UPEN

Career: Engineering in Information Technologies and Digital Innovation

Subject: Object-Oriented Programming

Activity: Justification of the sequence diagram

Student: Alex Antonio Pinto Cataño

Delivery Date: September 14, 2026

Justification of the sequence diagram

Reservation system for a sports center

1. The problem being modeled

The sports center offers various facilities (swimming pools, racquetball courts, gyms, and tennis courts). Member information is stored, including name, address, city, state, phone number, and membership fee. Additionally, items are available for reservation if needed (balls, nets, rackets).

Each facility is reserved by a member for a specific date, with a start and end time. A single reservation can include one or more sports items, and these are rented separately. For example, to play tennis, you must reserve the court and, if needed, the rackets as well.

This sequence diagram represents exactly that process: the complete, step-by-step journey of how a member makes a reservation.

2. Why was a sequence diagram chosen?

A sequence diagram is used to show the order in time that different elements of a system communicate with each other to complete a process. It is the correct tool here because the statement describes an action (booking), not just data.
The class diagram shows the structure of the data (what attributes each entity has), but it does not say in what order things happen.

The entity-relationship diagram shows how the tables in a database are related, but it does not represent the flow of a process.

The sequence diagram does answer the question that the statement actually asks: "What steps does the system follow when someone makes a reservation?"

3. The participants: why these and not others

Partner (actor): He is drawn with a stick figure because he is an actor, that is, someone external to the system who uses it. He is the one who initiates the whole process.
System: Represents the software that coordinates everything. Almost all decision logic (validate, calculate, confirm) resides here because it is the central component that connects the partner to the center's resources.

Facility: It is modeled as a separate object because it has its own independent state: its availability calendar. The System needs to "ask" the facility if it is free, so it must be a separate participant.

Sports equipment: For the same reason as the facility: it has its own inventory and availability (for example, how many rackets are free at that time), and the statement indicates that it is rented "separately", that is, with its own verification process.

4. Path and justification of each message

The following table explains where each step of the diagram comes from and why it has that shape (solid arrow, dotted arrow, or self-message).

Message	Justification (why it exists)	Notation used
StartReservation(date, starttime, endtime)	The statement reads: “Each installation is reserved for a specific date, from a start time to an end time.” These three pieces of information must be captured from the very first message because they are the basis of everything that follows.	Synchronous message (solid arrow) from the Partner actor to the System object.
validarMembresia()	The statement indicates that the system stores member data, including their membership fee. Before allowing a reservation, the system must confirm that the person requesting it is indeed a registered member.	Self-message (the arrow goes out and returns to the same activation bar) because it is an internal task of the System, it does not involve another object.
verifyCuota()	It is separate from membership validation because they are two different verifications: one confirms that the member exists, the other that	Self-message, same as the previous one.

	they are up to date with their payment.	
selectInstallation()	The center offers various facilities (swimming pools, racquetball courts, gyms, tennis courts). Members must indicate which facilities they wish to use before availability can be checked.	Self-message or internal step of the System that processes the choice already sent in the first message.
checkAvailability() (to Installation)	Although the statement does not say so literally, it is a mandatory logical consequence: if each reservation has a date and time, the system must necessarily check that no one else has that facility occupied at that same time.	Synchronous message from the System to the Installation object; activates its activation bar.
available: boolean	It is the mandatory answer to the previous question; without it the System could not decide whether to continue or reject the reservation.	Installation return message (dotted arrow) to System.
confirmInstallation()	It closes the first block of the process: it informs the partner that that part of their reservation (the facility) was secured.	System return message to the Partner actor.
requestArticle()	The statement explicitly says: “a reservation may include one or more sporting goods” and gives the example of tennis, where in addition to the court, you can request the racket.	Synchronous message from the Partner to the System; opens a new activation because it is a second independent request.
checkAvailability() (to Item)	The same reasoning applies as with the installation: the items are physical	Synchronous message from the

	and limited goods (there are only a certain number of rackets), so it is also necessary to check that they are available.	System to the Sports Article object.
available: boolean	Equivalent response to that of the installation, but for the requested item.	Return message from Sports Article to System.
calcularCosto()	The statement mentions a membership fee and items that are rented “separately”, implying that each item has an associated cost that must be added up before confirming.	System Self-Message.
registrarReserve()	A confirmed reservation must be saved so that it can be consulted later; this is a necessary step for the system to be functional, even though the statement does not detail it word for word.	System Self-Message.
confirmarReserva()	It is the natural closing of the process: the partner needs a final confirmation that their complete reservation (installation + items) has been made.	System return message to the Partner actor.

5. Which cases does the diagram cover and which does it not (and why)

Cases that are covered

- Book only one facility, without items (for example, just the pool).
- Book a facility and also one or more items separately (the example of tennis: court + rackets).
- Verification that the member has an active membership and their dues are up to date before allowing them to make a reservation.

Cases that are NOT covered (and why it's right that they aren't)

- Canceling an existing reservation: the statement only describes the process of creating a reservation, not its cancellation. That would be a different sequence diagram, with its own messages.

- What happens if the installation or item is unavailable? The diagram shown represents the "happy path" (everything goes well). The negative case would be modeled with a UML fragment called "alt" (alternative), showing what happens if the answer is false. It's acceptable not to include it if the problem statement doesn't explicitly require it, but it's important to be able to explain this difference if the instructor asks.

- Payment or renewal of the membership fee: the diagram only verifies that the fee is up to date, it does not model how the member pays that fee; that would be another separate process.

- To reserve several different items in a single reservation (e.g., racket and balls at the same time): the diagram shows a single requestItem() cycle, but in UML this could be repeated using a "loop" fragment to explicitly state that this message block is repeated once for each item ordered.

6. How to read the notation (to explain the symbols if asked)

- Stick figure = actor: someone external to the system (the partner), not a class within the system.
- Rectangle with underlined name = system object (System, Installation, Sports Item).

- Dotted vertical line = lifeline: represents the passage of time for that participant, from top to bottom.

- Narrow bar above the lifeline = activation bar: indicates the exact moment that participant is processing something.

- Solid arrow = synchronous message: a request, “I need you to do this”.
- Dotted arrow = return message: the response to a previous request.

- Loop arrow (goes out and returns to the same bar) = self-message: the participant sends a message to himself because it is an internal task that does not need anyone else.

7. Possible questions from the teacher and how to answer them

Why did you separate validateMembership() and verifyQuota() into two different messages?

A: Because they are two different validations: one confirms that the person is a member, the other that their payment is up to date. Separating them makes it clearer what is being checked at each step.

Why does `verifyAvailability()` appear twice, once for the installation and once for the item?

A: Because they are two independent resources with their own availability: the court may be free but the rackets may not, or vice versa. Each one needs its own verification.

What if the installation isn't available? Why didn't you draw it?

A: The diagram models the happy path, that is, when everything goes well, because that's what the statement describes. The 'unavailable' case would be represented with an alternative (alt) fragment in UML, which shows a different branch of messages when the condition is false.

Why does the circle with the stick figure represent the partner and not a rectangle like the others?

A: Because in UML that is the standard notation for an actor: someone external to the system that uses it, unlike objects (rectangles), which are internal parts of the system being designed.

Why did you add calculateCost() and registerReservation() if the statement doesn't mention them in those words?

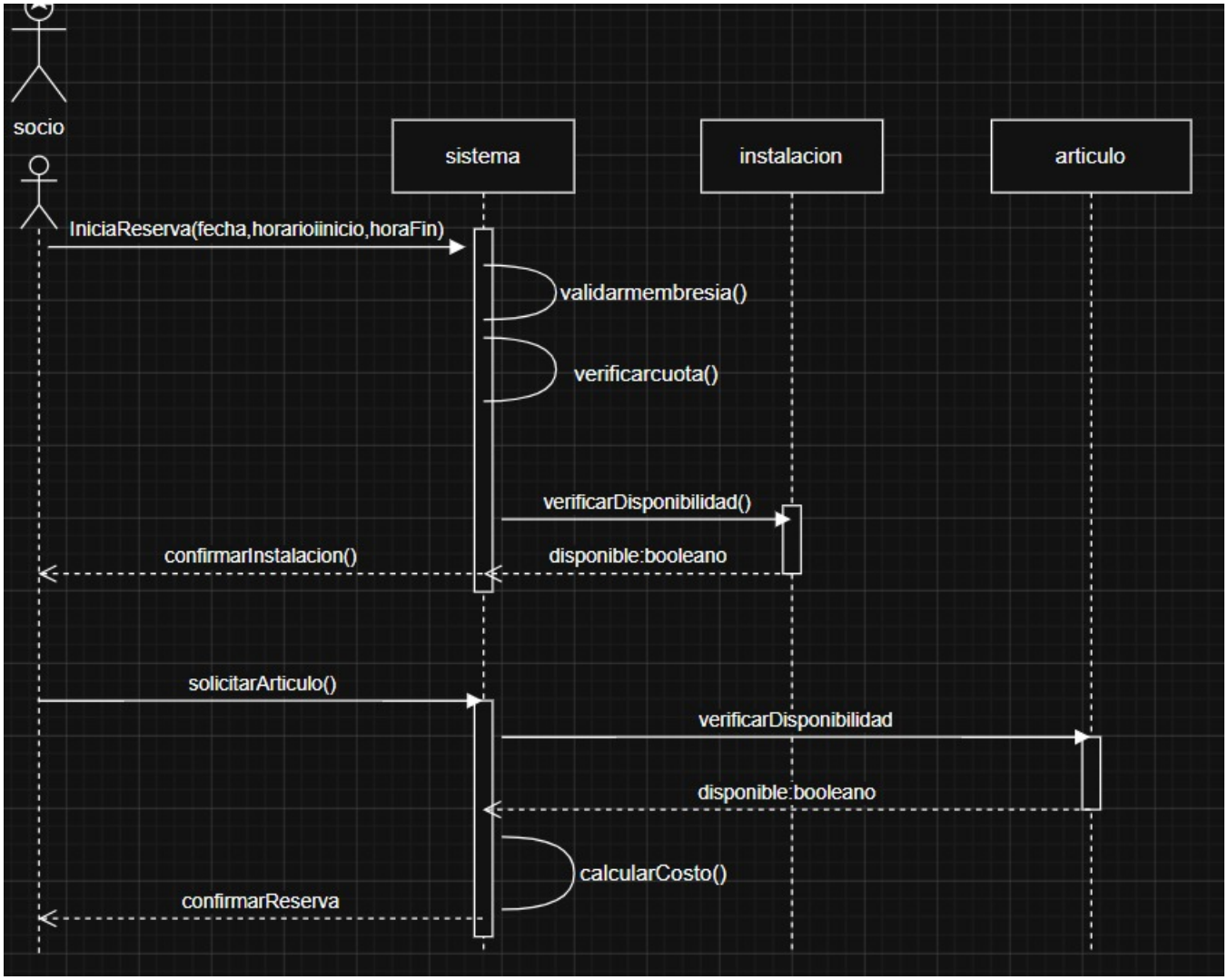
A: Because they are necessary logical consequences of the statement: if there is a membership fee and items that are rented separately, at some point those costs must be added together; and if the member receives a final confirmation, it is because the reservation was saved in the system.

Why does the system have two separate activation bars instead of a single continuous one?

A: Because they are two separate requests from the partner: first, they request the installation (and that system activity ends when it is confirmed), and then, at a later time, they request the item. These are two separate conversations, not a single continuous one.

Q: Does this diagram work for any facility, or just for tennis?

A: It works for any facility. The tennis example (court + rackets) was only used in the problem statement to illustrate the case where an item is needed; the diagram is general because requestItem() is optional and can be applied to any facility that requires an item, or simply omitted if none is needed.



Conclusion

Creating this diagram was like putting together a puzzle where we had to make important decisions to ensure everything was clear and worked well:

- **Step by step:** One of the biggest challenges was figuring out the exact order in which things happen. Unlike other diagrams that only show what information exists, this one

forced us to think in terms of actions: first requesting dates, then checking who the partner is, seeing if the court is available, and finally, collecting payment and confirming the reservation. We had to organize all of this logically.

- **Separating tasks: It was** important not to pile up the tasks. For example, at first it seemed like a good idea to check the partner in a single step, but then we decided it was better to divide it into two: first, to see if they actually exist in the system, and then to check if they've Already paid. Your monthly payment. Doing it in parts makes the process easier to understand and correct if something goes wrong.

- **Multiple checks:** Another challenge was understanding that a reservation can have separate parts. It's not enough to check if the tennis court is available; if the member also requested rackets, you have to check the racket inventory separately. We realized that the court and the equipment are different things, and each requires its own check.

- **Deciding what to show (and what not to):** Ultimately, we had to limit the diagram to avoid making it too large. We decided to show only the perfect scenario (the "happy path"), where everything goes smoothly and nothing goes wrong. We deliberately left out what would happen if the court is occupied or how the member pays their monthly fee, so that the diagram would focus strictly on the main process of making a reservation.